Web 2 – Séance 07

Projet Vite et composant React

React

- Bibliothèque JavaScript pour construire des interfaces utilisateur
- **Déclaratif**
  - Le développeur décrit l'UI
  - React gère les mises à jour du DOM
- Basé sur les **composants**
  - Pièces réutilisables d'interface
  - Unités autonomes

Vite

- Outil de build pour les applications web modernes
- Permet de créer un projet React + TypeScript en quelques commandes
- Transpile le code TypeScript/JSX en JavaScript compatible navigateur
- Propose un serveur de développement avec hot reload
- Rassemble l'entièreté du code source en un bundle optimisé pour la production
  - Un seul fichier HTML
  - Un seul fichier CSS
  - Un seul fichier JavaScript avec code minifié
  - Fichiers statiques optimisés (images, fonts, etc.)

Créer un projet Vite + React + TypeScript

```
npm create vite@latest
```

- Crée un projet préconfiguré avec Vite
- Pose des questions pour choisir :
  - le nom de projet
  - le framework → React
  - le langage → TypeScript
  - le linter → Oxlint

```
npm create vite@latest frontend -- --template react-ts --no-eslint
```

- Choix préconfigurés pour le cours :
  - `frontend` : nom du projet et du dossier créé
  - `--` : sépare les options de npm de celles transmises à Vite (obligatoire)
  - `--template react-ts` : React avec TypeScript
  - `--no-eslint` : utilise le linter Oxlint

Structure du projet

```
frontend/
├── public/              # Fichiers servis tels quels (favicon)
├── src/
│   ├── assets/          # Images importées par le code
│   ├── App.tsx          # Composant racine de React
│   ├── App.css          # Styles du composant App
│   ├── index.css        # Styles globaux
│   └── main.tsx         # Point d'entrée javascript, monte le composant App à l'élément #root
├── .gitignore           # Fichiers ignorés par Git
├── .oxlintrc.json       # Config du linter
├── index.html           # HTML root, contient <div id="root"></div> pour React
├── package.json         # Dépendances et scripts
├── package-lock.json    # Versions exactes des dépendances
├── README.md            # Documentation du projet
├── tsconfig.app.json    # Config TypeScript pour le code React
├── tsconfig.json        # Config TypeScript, séparé en deux sous-fichiers
├── tsconfig.node.json   # Config TypeScript pour le code Node.js
└── vite.config.ts       # Config Vite, rien à modifier pour l'instant
```

Commandes du projet

- `npm install` : installe les dépendances du projet
- `npm run dev` : lance le serveur de développement Vite
  - Port par défaut : 5173 (http://localhost:5173)
  - Compile le code TypeScript/JSX à la volée
  - Recharge automatiquement la page dans le navigateur quand un fichier est modifié
- `npm run build` : compile le projet pour la production
  - Compile le code TypeScript/JSX en JavaScript compatible navigateur
  - Minifie le code et les fichiers CSS
  - Copie les fichiers statiques optimisés dans le dossier `dist/`
  - Serveur de production : serveur web de fichiers statiques avec le contenu du dossier `dist/`

JSX: HTML en JavaScript/TypeScript

Fichiers `.tsx` = TypeScript + JSX. Permet d'écrire du HTML-like dans le code TypeScript.

```tsx
const element: JSX.Element = <h1>Bienvenue!</h1>;

// Avec des variables
const title = "Recettes";
const greeting = <h2>{title}</h2>;

// Avec des fonctions

const info = <p>Durée: {formatDuration(duration * 60)}</p>;

// Avec des conditions (ternaires)
const badge = (
  <span>
    {difficulty === "facile" ? "✅" : "⚠️"}
  </span>
);
```

JSX: Différences avec HTML

- `className` au lieu de `class` (mot-clé réservé JS)
- `htmlFor` au lieu de `for` (mot-clé réservé JS)
- Expressions JavaScript entre `{}`
- Le texte inséré avec `{}` est **échappé** : `{"<img src=x onerror= ... >"}` s'affiche tel quel, comme avec `textContent` → React protège ainsi contre les attaques XSS (séance 05)
- `style` reçoit un objet, pas une chaîne : `style={{ width: "200px" }}`
- Tags auto-fermants obligatoires : `<img />`, `<input />`, `<br />`
- Une expression JSX a **un seul élément racine** : envelopper dans un `<div>` ou un fragment `<> ... </>`
- Commentaires : `{/* ... */}`

Composants React

Un composant React est une fonction TypeScript qui retourne du JSX

```
const Welcome = () => {
  return <h1>Bienvenue dans MiamMiam!</h1>;
};

// Avec du contenu plus complexe
const RecipeCard = () => {
  return (
    <div className="recipe-card">
      <h2>Pâtes Carbonara</h2>
      <p>Durée: 20 minutes</p>
      <p>Difficulté: Facile</p>
    </div>
  );
};
```

- **Naming**: Commencez par une majuscule = composant React
- **Structure**: Une fonction = un composant

Exemple: RecipeCard

```tsx
// RecipeCard.tsx
const RecipeCard = () => {
  return (
    <div className="recipe-card">
      <h2>Pâtes Carbonara</h2>
      <img
        src="https://images.unsplash.com/photo-1612874742237-6526221588e3?w=800"
        alt="Pâtes carbonara"
        style={{ width: "200px", height: "150px", objectFit: "cover" }}
      />
      <p>
        <strong>Durée:</strong> 20 minutes
      </p>
      <p>
        <strong>Difficulté:</strong> Facile
      </p>
      <p>
        <strong>Ingrédients:</strong> Pâtes, œufs, lard, fromage
      </p>
    </div>
  )
};

export default RecipeCard;
```

Utiliser un composant

Importer et rendre le composant comme une balise JSX

```tsx
// App.tsx
import RecipeCard from "./components/RecipeCard";

const App = () => {
  return (
    <div className="app">
      <h1>MiamMiam</h1>
      <RecipeCard />
    </div>
  );
};

export default App;
```

Utiliser un composant (suite)

Fichier généré par Vite, à ne pas modifier pour l'instant :

```tsx
// main.tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.tsx";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

- `StrictMode` : mode de développement qui signale les erreurs courantes

Plusieurs composants : un fichier

```tsx
// App.tsx

const Header = () => <header>MiamMiam</header>;
const Footer = () => <footer>© 2026</footer>;

const App = () => (
  <div>
    <Header />
    <main>Contenu</main>
    <Footer />
  </div>
);

export default App;
```

- Un composant principal, avec un affichage décomposé en sous-composants
- Un seul composant exporté par fichier

Plusieurs composants : plusieurs fichiers

```
src/
├── App.tsx
├── components/
│   ├── RecipeCard.tsx
│   ├── Header.tsx
│   └── Footer.tsx
```

- Un fichier par composant dès qu'il est réutilisé ailleurs ou qu'il devient long
- Dossier `components/` pour tout ce qui n'est pas `App`
- Nom du fichier = nom du composant

Récapitulatif

- **React** – bibliothèque pour construire des interfaces utilisateur
- **Vite** – bundler moderne pour compiler et servir le code React
- **JSX** – syntaxe HTML-like dans le code TypeScript
- **Composants** – fonctions TypeScript qui retournent du JSX, organisés en fichiers

Prochaine séance : Séance 08 – Props et children

Exercice filé S07

1. Créez l'application `frontend` dans votre dossier d'exercices MiamMiam
2. Créez un fichier `RecipeCard.tsx` dans le dossier `src/components`
3. Dans ce fichier, créez l'objet `recipe` suivant :

```
const recipe = {
  title: "Pâtes Carbonara",
  imageUrl: "https://images.unsplash.com/photo-1612874742237-6526221588e3?w=800",
  duration: 20,
  difficulty: "Facile",
};
```

4. Toujours dans ce fichier, créez un composant `RecipeCard` qui affiche cette recette avec :
   - Son titre à l'aide d'une balise h2
   - Son image à l'aide d'une balise img
   - Sa durée et sa difficulté à l'aide de balises p

Exercice filé S07 (suite)

5. Enlevez le contenu de `App.tsx` et remplacez-le par le code suivant :

```
const App = () => {
  return (
    <div className="app">
      <h1>MiamMiam</h1>
    </div>
  );
};

export default App;
```

6. Intégrez le composant `RecipeCard` dans `App.tsx` pour qu'il s'affiche sous le titre
7. Intégrez un deuxième composant `RecipeCard` dans `App.tsx`
8. Vérifiez que les deux cartes s'affichent correctement dans le navigateur

Exercice complémentaire EC03

1. Téléchargez la page HTML de la séance 07 sur moodle et ouvrez-la dans le navigateur
2. Créez un projet `EC03` dans votre dossier d'exercices complémentaires
3. Sur papier, découpez la page en différents composants
4. Créez les composants et assemblez-les dans le projet pour reproduire la page à l'identique
5. Copiez le CSS de la page dans `src/index.css` (le contenu généré par Vite peut être supprimé)
6. Vérifiez que la page s'affiche comme l'originale